

PART VI — THE SELF-LEARNING LAYER

Chapter 33

Hybrid Retrieval Over Documents and Learned Memory

“Two collections that mean different things cannot be searched as if they were one, no matter how similar their vectors look.”

Chapter 33 — Hybrid Retrieval Over Documents and Learned Memory

This project's own retrieval design has a real, documented history worth teaching in order rather than presenting only the current end state. ADR-016 first introduced `learned_qa` as a second collection with a simple mandate: "retriever.py implements hybrid retrieval across both collections" — one merged, re-ranked list. Later investigation (Research topic 26) found that single-pool merge had structural pathologies severe enough to motivate a full redesign into the two-track parallel retrieval Chapter 22C already covers from the compression side. This chapter covers it from the retrieval side, and adds the scoring precision — distance-metric consistency, per-collection thresholds, per-collection top-k — that makes either design trustworthy.

33.1 Two collections, one retriever

The original design was architecturally simple: one `RAGRetriever`, one `retrieve()` method, querying both the `documents` and `learned_qa` ChromaDB collections with the identical query embedding and combining whatever came back. This is the natural first design for hybrid retrieval — it requires no new data structures, no precedence rules, and no changes to any downstream code that already expected "a list of chunks" from retrieval.

The appeal is real: a single merged list is exactly what every consumer downstream of retrieval — the compression pipeline, the validators, the final answer generation — was already built to expect from Chapter 11 onward. Introducing a second collection without changing that shape looks, at first, like a free improvement.

33.2 Merging, deduplicating, and re-ranking across collections

The merge itself is straightforward: query both collections, tag each result with its origin collection, sort the combined list by `similarity_score` descending, deduplicate by chunk ID, and truncate to the configured `top_k`. `RAGRetriever._rank_collection_results()` — the method this project's current code still uses for within-collection ranking — is a direct descendant of exactly this merge-and-truncate logic, now applied per collection instead of across both at once.

Deduplication in a merged pool is more subtle than deduplication within a single collection, because `documents` and `learned_qa` entries can never share an ID by construction — one is chunk-derived, the other is a SHA-256 content hash (Chapter 32.5) — so ID-based dedup alone cannot catch the case where a learned-QA pair and its own source document chunk both appear in the same merged result, saying essentially the same thing in different words.

33.3 Weighting learned memory vs raw documents

A single merged, score-sorted list has no natural place to express a preference between the two collections — score is score, and a `similarity_score` of 0.71 from `documents` outranks a `similarity_score` of 0.68 from `learned_qa` by the sort alone, regardless of whether learned memory should, in principle, be trusted more for a query it directly matches. Weighting schemes to compensate — multiplying one collection's scores by a boost factor before the merge, for instance — are a real option, but they push an implicit precedence decision into the same numeric space as relevance ranking, where it becomes difficult to reason about independently of the actual similarity math.

This is precisely the structural gap Research topic 26 identifies as the deeper problem: "validators that received mixed lists having no place to express precedence." A validator judging a merged list's relevance cannot also apply "prefer learned QA on conflict" as a separate, later decision — by the time the list is merged, the information about which collection each entry came from has to be carried alongside every entry just to make that later decision possible at all, and every downstream consumer has to know to look for it.

33.4 Updating retriever.py for hybrid behavior

The original `retrieve()` method's job was exactly this merge: query, tag, sort, truncate, return one list. Compare this project's current `retrieve()` — which now queries `documents` only, having been superseded for hybrid use by `retrieve_separate()` — and the shape of the change is visible directly in the code that remains.

```
def retrieve(self, query, top_k=RETRIEVAL_TOP_K, score_threshold=0.0):
    """Retrieve from the documents collection only."""
    query_embedding = self._embed_and_log(query)
    results = self._query_collection(self.vector_store.collection,
    query_embedding, top_k)
    documents = self._rank_collection_results(results, limit=top_k,
    score_threshold=score_threshold)
    self._last_document_chunks = documents
    self._last_learned_qa_chunks = []
    return documents
```

The docstring — "Retrieve from the documents collection only" — is itself a small piece of architectural history: a method that once merged two collections now explicitly disclaims doing so, its hybrid responsibility fully migrated to a sibling method built around two independent result lists instead of one merged one.

33.5 Watching the learned collection grow over time

A hybrid retriever's behavior is not static — `learned_qa` starts empty and grows with every distillation pass Chapter 32 triggers, which means the *same* retrieval code produces measurably different results on day one versus month three of a deployment's life. Early on, with few or zero learned entries, hybrid retrieval degrades gracefully to something close to document-only retrieval; `retrieve_separate()`'s own guard — `if self.learned_collection and self.learned_collection.count() > 0` — exists specifically so querying an empty or absent learned collection is a no-op rather than an error.

As the collection grows, an increasing share of queries plausibly match something in `learned_qa` directly, and Chapter 22C.10's precedence rule starts mattering in practice rather than only in principle. This growth curve is worth monitoring explicitly — Chapter 35.2's `stats` command surfaces the raw count precisely because "how many entries are in `learned_qa`" is a meaningful operational signal, not merely a curiosity, for understanding why a deployment's retrieval behavior in month six differs from its behavior in week one.

33.6 Hybrid single-pool vs. two-track parallel retrieval

Research topic 26 frames the actual design decision precisely: source-document chunks are "ground-truth-by-provenance," while learned-QA chunks are "user-validated synthesis" — two qualitatively different kinds of evidence with different conflict-resolution needs, which "cannot be made by a cosine-similarity ranker that doesn't even know which collection a chunk came from." The single-pool design of Sections 33.1-33.4 collapses that distinction the moment the merge happens; two-track parallel retrieval (`retrieve_separate()`, Chapter 11.5) preserves it all the way through the pipeline.

Figure 33.1 traces the single-pool merge through to its actual failure point — not the query or the individual collection searches, both of which work exactly as intended, but the truncation step, where a fixed `top_k` applied to one combined list has no way to know it is silently discarding a collection's worth of evidence rather than merely trimming excess.

The original hybrid single-pool merge — and its displacement risk

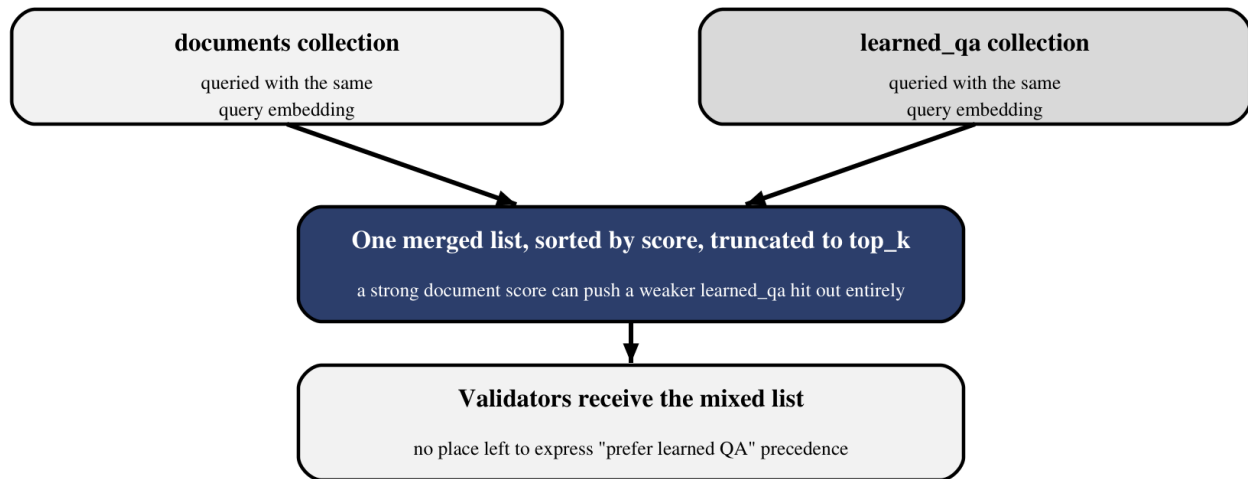


Figure 33.1 — A single merged, truncated list has no room left to express which collection a surviving chunk came from, or to protect a weaker learned-QA hit from being crowded out by a stronger document score.

Property	Single-pool merge	Two-track parallel
Downstream shape	One list — no code changes needed elsewhere	Two lists — every consumer must handle both
Precedence expression	Requires per-entry origin tagging, easy to lose	Native — each track is independently addressable
Truncation risk	Strong documents can crowd out weak learned QA	Each track keeps its own top_k / top_1 budget
Validator design	One validator judging a mixed, ambiguous list	One validator per track, per collection's own norms

The conclusion this project actually reached, per Research topic 26, was unambiguous: "two-track retrieval is the right architecture for self-learning RAG," with each channel "queried independently... validated independently, deduplicated and merged independently, accumulated independently, compressed independently, and combined only at the LLM context boundary with an explicit conflict-resolution rule." This chapter's remaining sections apply to *either* design equally — the scoring precision below matters whether the two collections' results ultimately merge into one list or stay in two.

33.7 Distance-metric consistency across collections

A subtler failure than the merge-and-truncate problem hides directly in the similarity math: `'documents'` was created with `'hnsw:space = cosine'`, but `'learned_qa'` was created without specifying `'hnsw:space'` at all, silently defaulting to ChromaDB's L2 distance. Both collections were scored with the identical formula, `'similarity_score = 1 - dist'`, and filtered through the identical `'MIN_SIMILARITY'` threshold — but `'1 - dist'` means genuinely different things depending on which metric actually produced `'dist'`.

On unit-normalized embeddings (which `'all-MiniLM-L6-v2'` produces), the relationship is $\|a - b\|^2 = 2 - 2 \cdot \cos(\theta)$, so `'1 - dist'` under cosine is true cosine similarity, while `'1 - dist'` under L2 works out to $\sqrt{2} \cdot \cos(\theta) - 1$ — the same ranking order, but a completely different threshold meaning. A learned-QA chunk that just cleared a 0.5 floor under L2 corresponds to a true cosine similarity around 0.75; strongly relevant, filtered as if it were only marginally so.

Figure 33.2 makes the mismatch concrete: the same formula, the same numeric floor, applied to two collections whose underlying distance semantics were never actually the same thing, producing a threshold that is simultaneously correct for one collection and badly miscalibrated for the other, with no error message anywhere to indicate which.

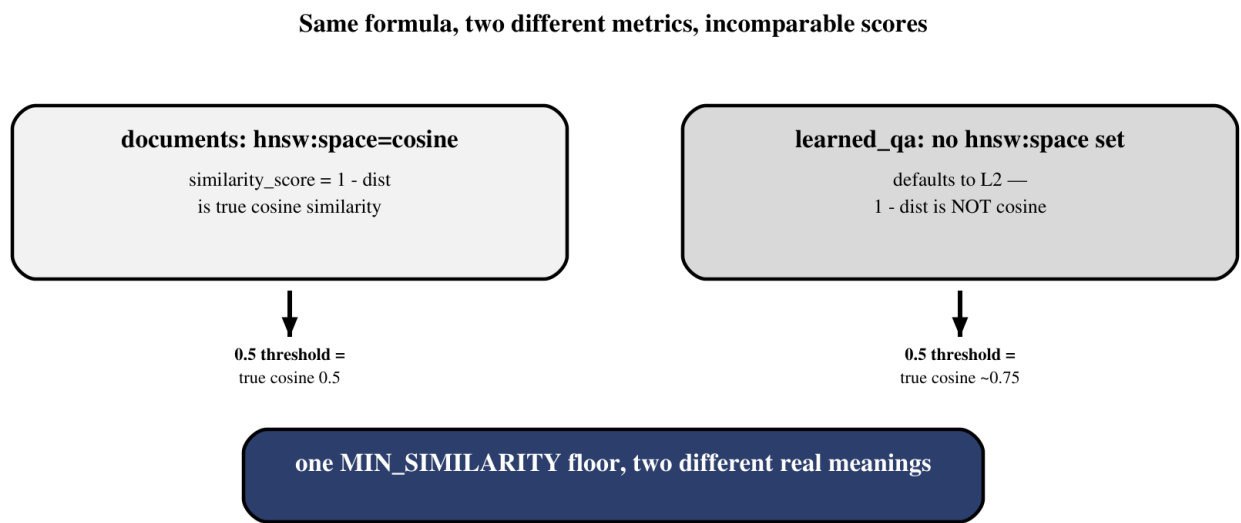


Figure 33.2 — The identical `1 - dist` formula and the identical 0.5 floor mean genuinely different relevance thresholds depending on which collection's distance metric produced the score.

This is BUG-030 in this project's own ledger, and its fix required more than a metadata edit: ChromaDB's `'get_or_create_collection()'` does not migrate an existing collection's distance metric, so correcting `'learned_qa'` to cosine required a full snapshot-and-restore of the collection's HNSW index — a 374-entry live migration, count-verified, with rollback on failure. The remediation pattern that followed — funnel all collection creation through a single factory, and detect non-canonical distance metrics at startup — is the general lesson: any two collections sharing scoring infrastructure must be created through code that enforces metric consistency at the single point of creation, not assumed to follow from documentation or convention.

The root cause is worth stating plainly because it is a genuinely easy trap: `'documents'` was created early in the project with an explicit `'hnsw:space="cosine"'` argument, a choice made deliberately at the time. `'learned_qa'` was created later, by different code (`'self_learner.py'`'s `'get_or_create_learned_qa_collection()'`), written without anyone re-checking whether the same explicit argument had been carried over. ChromaDB's silent fallback to L2 when the argument is simply absent means this kind of drift produces no error, no warning, and no visible symptom until someone specifically investigates why learned-QA relevance filtering behaves inconsistently with document filtering under an identical threshold value.

33.8 Per-collection score thresholds

Once the metric-consistency problem is fixed, a second, independent question remains: should `documents` and `learned_qa` even use the **same** similarity floor, correctly computed or not? This project's evidence-based answer, from Research topic 39's log analysis, was no — the two collections' actual score distributions differ enough to warrant separate floors. Document chunks, retrieved from source PDFs and technical text, showed relevant matches clustering lower; learned-QA entries, being syntactically closer to natural queries by construction (they are themselves distilled question-answer pairs), clustered higher.

Constant	Evidence-based value	Why it differs from the other collection
DOCUMENTS_MIN_SIMILARITY	0.53	Source-chunk relevance clusters lower; a tighter floor risks losing real matches
LEARNED_QA_MIN_SIMILARITY	0.57	Distilled Q&A pairs read closer to natural queries; irrelevant entries score visibly higher, needing a stricter floor

A single shared `MIN_SIMILARITY` — the pre-split default — necessarily gets one of the two collections wrong: set for documents, it under-filters learned QA; set for learned QA, it over-filters documents. Splitting the constant in two is not a tuning nicety, it is the only way for one number to correctly represent two collections whose underlying score distributions genuinely differ.

33.9 Per-collection top-k

`RETRIEVAL_TOP_K` (documents) and `RETRIEVAL_TOP_L` (learned_qa) exist as two separate constants for the identical reason the thresholds split: `retrieve_separate()` queries each collection with its own depth parameter, independently of how many results the other collection returns. This project's own evidence-based tuning (Research topic 39, later refined by the A/B methodology Chapter 36C covers in full) converged on `RETRIEVAL_TOP_K=4` and `RETRIEVAL_TOP_L=4` — equal in this specific case, but arrived at independently rather than assumed equal by default, and free to diverge again if either collection's evidence changes.

Two-track parallel retrieval is what makes independent top-k values meaningful in the first place — in the single-pool design of Sections 33.1-33.4, one shared `top_k` applied to the **merged** list, meaning a change to "how many documents chunks to fetch" and "how many learned-QA chunks to fetch" could not be tuned separately at all. This is one more concrete way Section 33.6's architectural choice pays for itself: not just cleaner precedence handling, but a retrieval-depth knob per collection that the single-pool design structurally could not offer.

Chapter 34 turns to a problem this chapter's own merge-and-dedup discussion already previewed: when retrieval runs multiple times across a single agentic loop's iterations, near-duplicate chunks accumulate even within one collection's own results, and the deduplication and merging machinery needed to handle that is its own careful piece of engineering.

It is worth closing by naming what this chapter's two case studies — the single-pool-to-two-track redesign and the distance-metric fix — have in common. Neither was visible from reading `retriever.py` in isolation; both required watching the system under real, repeated use and noticing a pattern a single test case would never surface. A displaced learned-QA chunk in one query looks like an unlucky ranking outcome. A misfiled distance metric in one collection looks like a marginally under-performing threshold. Only at the scale of many runs, examined together, did either pattern reveal itself as a structural problem rather than noise — which is exactly the evidence-based tuning discipline Chapter 36C generalizes into a repeatable methodology.